

Name: Ngeno Kibet Caleb

Registration number: SCT221-C004-0755/2017

Unit name: OBJECT-ORIENTED PROGRAMMING I

Unit code: ICS2104

CAT

1.

a. Define the following object-oriented concepts

- i) class is a collection of objects that have similar properties binding data and functions together.
- ii) object is an instance of a class possessing the attributes and methods defined in the class.
- iii) inheritance is the ability to define new classes from existing classes promoting code reuse and hierarchical relationships between classes.
- iv) encapsulation is the practice of enclosing data (attributes), methods (functions) within a class and access to the internal state of an object using modifiers like private and public.

b. With aid of examples, show how each of the following object-oriented concepts are implemented. i) object ii class iii) encapsulation

Scenario

object – myCar (brand, color)

class – Car

encapsulation – personal identification information of the owner

```

#include <iostream>
using namespace std;
//class
class Car {
private: // Encapsulation
    string ownerID;
    void showOwnerInfo() { // Private method
        cout << "Owner ID (protected): " << ownerID << endl;
    }
public:
    string type;
    Car(string t, string id) : type(t), ownerID(id) {} // Constructor
    void displayInfo() { // Public method
        cout << "Car Type: " << type << endl;
        showOwnerInfo();
    }
}
//object
int main() {
    Car myCar("electric", "sedan");
    myCar.displayInfo();
    return 0;
}

```

Code

c. State any TWO characteristics of identifiers

- unique identification
- valid naming rules

d. Create a class called rectangle with two integer data members, i.e. length and width. The class should also have constructor that initializes the two data members with values. The class should also have a member function that computes and outputs the area and perimeter of rectangle.

```

#include <iostream>
using namespace std;

class Rectangle {
public:
    // initialize length and width
    Rectangle(int l, int w) {
        length = l;
        width = w;
    }
    // area
    void computeArea() {
        int area = length * width;
        cout << "Area of rectangle: " << area << endl;
    }
    // perimeter
    void computePerimeter() {
        int perimeter = 2 * (length + width);
        cout << "Perimeter of rectangle: " << perimeter << endl;
    }
};

```

2. a) i) State any TWO properties of friend functions

- not a member of class
- access to private members of the class

ii)

```
class ClassName {  
private:  
    int data;  
public:  
    friend void functionName(ClassName &obj);  
};
```

b. With the aid of examples, differentiate relational operators and logical operators

relational operators - $a < b$ (2 comparisons)

logical operators – $a < 0 < b$ (combine more than 2 comparisons)

Both have true/false Boolean return types along with comparatives ($<$, $>$, $=$)

c. Using examples, differentiate primitive data types and reference data types

Primitive data types are built-in in a programming language like float, int whereas reference data types are dynamic memory locations for variables and objects in object-oriented programming languages for instance class, object, arrays.

d. Using examples, differentiate between static variables and instance variables.

Static variables remain the same are declared by the word “static” while instance variables specific to objects.

```
class Counter {  
public:  
    static int count; // static variable  
    string name; // instance variable  
    Counter () count++;  
};
```

e. Show how each of the following keywords are used i) register ii) auto iii) extern iv) static

```
void demoStatic() {  
    static int x = 0;
```

```

auto x++;

cout << "Static x = " << x << endl;

};

extern int count;

int main () {

    cout << count;

};

register int counter = 0;

for (int i = 0; i < 3; i++) {

    counter += i;

};

```

3. a. Differentiate parameter and argument. Give examples.

Parameters are variables declared in the function header that act as placeholders for values whereas arguments are actual values in a function call.

```
void function(parameter)
```

```
function("Argument")
```

```
void multiply(a,b)
```

```
multiply (1,2)
```

b. State any TWO properties of a constructor

- Same name as defined class
- No return type

c. Show the syntax of declaring a structure and a union.

Structure is declared using keyword “struct” and union is declared using keyword “union”.

```
union Data {
```

```
    int intVal;
```

```
    float floatVal;
```

```
char charVal;  
  
};  
  
struct StructureName {  
  
    dataType member1;  
  
    dataType member2;  
  
};
```

d. Differentiate while loop and do...while loop.

While loop checks the condition before executing the body whereas do...while loop executes the body first, then checks the condition.

e. Write a program that uses loops to print out the pattern shown below

```
1  #include <iostream>  
2  using namespace std;  
3  
4  int main() {  
5      for (int i = 1; i <= 5; i++) {  
6          for (int j = 5; j >= i; j--) {  
7              cout << i << " ";  
8          }  
9          cout << endl;  
10     }  
11     return 0;  
12 }
```